

L2-3.1. Elastic: Query Languages

Task 1- Introduction

In a Security Operations Center (SOC), analysts are constantly inundated with data from various sources, including network traffic logs, intrusion detection systems, vulnerability scanners, and endpoint security software. Effectively sifting through this massive amount of information can overwhelm any analyst. Mastering advanced queries can significantly streamline this process, enabling analysts to extract critical insights and make well-informed decisions.

Task 2 - Elastic Languages and Syntax

In this task, we'll review the supported query syntaxes and clarify key concepts that will help you avoid mistakes when building complex searches.

Supported Syntaxes:

- [Lucene\(opens in new tab\)](#) is a powerful query syntax built on the Apache Lucene library, the core search engine underlying Elasticsearch. It supports advanced search capabilities, including pattern matching and positional queries.
- [Kibana Query Language\(opens in new tab\)](#) (KQL) is a simpler, user-friendly query language built specifically for filtering data in Kibana. KQL is ideal for quickly narrowing down results in Discover and dashboards. Please note that the Kibana Query Language is different from Microsoft's Kusto Query Language, despite sharing the same acronym (KQL).
- [Elasticsearch Query Language\(opens in new tab\)](#) (ES|QL) is a newer, pipe-based query language designed for structured data analysis in Elasticsearch. It allows you to filter, transform, and aggregate data using a sequential pipeline syntax, making complex queries easier to read and write. You might compare it with Splunk's SPL.

Lucene Example	KQL Example	ES QL Example
<code>flag: thm</code>	<code>flag: thm</code>	<code>FROM index-name WHERE flag == "thm"</code>

Special Characters: KQL is generally more forgiving and allows you to wrap values in quotation marks to treat them as literals. Lucene, however, is stricter and often requires explicit escaping with a backslash `\`. Failing to account for these differences can result in errors or unexpected results.

Special characters (`+ - && || ! ( { [ ^ ~ * ? : /`) are interpreted as operators and must be wrapped in quotes or escaped with a backslash in both Lucene and KQL. One example is searching for the literal flag value `THM{kql!}`:

- Quotes: `flag: "THM{kql!}"`
- Backslash: `flag: THM{kql!\}`

Quotation marks (`" "`) are used to define exact phrases and values with spaces. If you need to search for a literal quote character, escape it with a backslash.

Below are a few examples for the field `log_message`:

- Values with spaces: `log_message: "Authentication failed"`
- Quote character escape: `log_message: "User \"bob\" created"`

Periods (`.`) are commonly used for field names, such as `host.name`. When searching within string values, ensure it is interpreted correctly. For example, you may want to search for the phrase `.exe` in the `file.extension` field:

- KQL: `file.extension: ".exe"`
- Lucene: `file.extension: \.exe`

Wildcards: Wildcards allow you to search for partial matches within a field value, and are useful when you do not know the exact value or when multiple variations exist. The **multi-character wildcard** `*` matches zero or more characters. In the example below, we use the wildcard search `article_title: *Malware*`, which returns two results, both containing the term `Malware` within the field value.

The screenshot shows the Elasticsearch Kibana interface. At the top, the query bar contains 'article_title: *Malware*' and a 'Your Query' label. Below the query bar, there's a 'boosting' section with a search field for field names and a filter by type dropdown. The 'Returned Results' section shows two hits. The first hit has 'article_title: Malware Analysis Techniques' and 'article_tags: Malware'. The second hit has 'article_title: A Guide to Reverse Engineering Malware' and 'article_tags: Malware, Reverse Engineering'.

Field Types and Tokenization: Elasticsearch stores fields using different data types, most commonly `text` and `keyword`. Elasticsearch analyzes `text` fields, meaning the value is broken into smaller pieces, called tokens, during indexing. For example, the field value `Elastic Query Languages` will be split into the tokens `Elastic`, `Query`, and `Languages`. Alternatively, `keyword` fields are not analyzed, and the entire value is stored exactly as written. In this case, the field value `Elastic Query Languages` will be seen strictly as `Elastic Query Languages`.

Effect on Wildcard Searches: Because of this indexing behavior, wildcard searches do not always work as expected. On `text` fields, wildcards operate on individual tokens rather than the original full value, which means you cannot match across spaces, and partial matches may fail. Wildcard queries can also be slower, especially those that begin with `*`, because Elasticsearch must scan many indexed terms.

Answer the questions below

The value `david: davidson` exists in the `user.name` field.

Since the colon `:` is a special character in KQL, what query would you use to search for this exact value?

`user.name: " david: davidson "`

Suppose you want to find values such as `hack` and `hacking` in the `activity` field.

What wildcard query would return these results?

`activity: hack*`

Task 3 - Navigating Nested Data

Sometimes, fields in a dataset contain structured data rather than simple text values. In the example below, the fields `affected_systems` and `comments` store multiple objects in a JSON-like format. This allows us to search not just the field itself, but specific attributes inside those objects.

record_id	incident_type	affected_systems	comments
1	DDoS	[{system: web-server}, {system: database}]	[{author: Alice, text: Mitigated DDoS attack}, {author: Bob, text: Checked logs, found suspicious IPs}]
2	Malware	[{system: web-server}, {system: file-server}]	[{author: Charlie, text: Removed malware}, {author: Eve, text: Updated antivirus software}]
3	Data breach	[{system: database}]	[{author: Alice, text: Patched vulnerability}, {author: Eve, text: Reset all user passwords}]
4	Phishing	[{system: email-server}]	[{author: Bob, text: Blocked phishing email}, {author: Charlie, text: Sent warning to all users}]
5	Insider threat	[{system: file-server}, {system: database}]	[{author: Eve, text: Investigating employee activity}, {author: Alice, text: Implementing stricter access controls}]

In the above dataset, the `comments` field is an array of objects, each with an `author` and a `text` field. As an example, imagine you want to return all entries with any value in the `comments.author` field. We could use the `*` wildcard as we've learned in the previous task with the query `comments.author: *`.

- This would return all entries from 1 to 5 since `*` matches any value after `comments.author`.

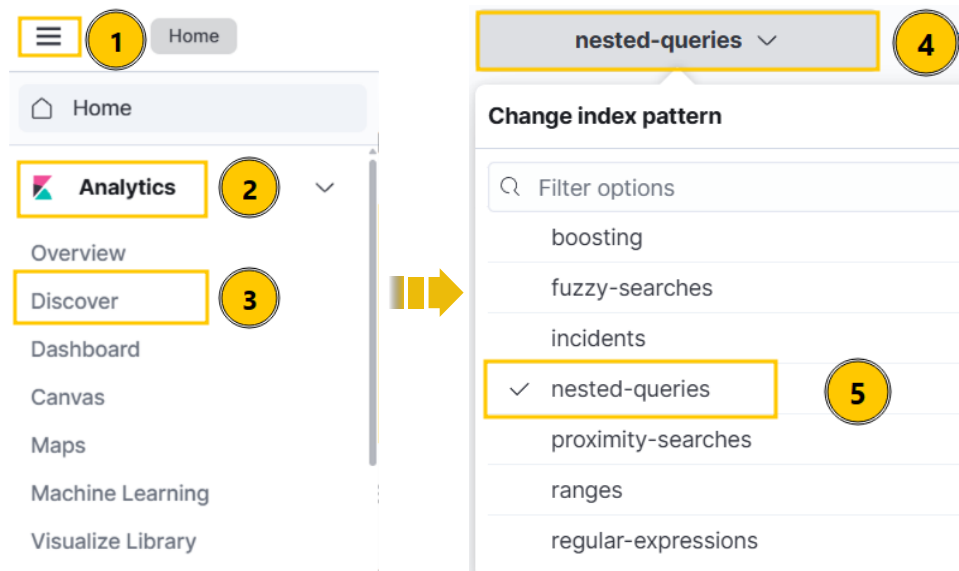
Perhaps next we want to search the `comments.author` field for values that only contain `Alice`. We can use the query `comments.author: "Alice"`.

- This will return records 1, 3, and 5, as these entries have `Alice` as the `author`.

Nested Data in Discover

Let's jump into our Elastic instance and try querying nested data. Go ahead and access the Kibana menu in the top left from the Kibana homepage.

1. Click the **Kibana** menu
2. Highlight the **Analytics** section
3. Click **Discover**
4. Once on the **Discover** homepage, click to choose an index pattern
5. Select `nested-queries`



Above, we discussed a couple of different queries we could use to highlight specific information within nested data. Let's try out the query below to locate records written only by `Alice` that contain the word `attack` in the `comments.text` field. We can combine two queries with the `AND` operator.

1. Ensure you've selected the `nested-queries` index pattern
2. Enter the query `comments.author: "Alice" AND comments.text: "attack"`
3. View the results with both `Alice` and `attack` highlighted



Grouping Values Within a Single Field: In the query above, we used the **AND** operator to search for two terms in different fields. However, there are instances when we want to search for multiple values in the same field. In this case, we use parentheses **()** to group values. In the screenshot above, you can see that there are two names within the **comments.author** field. We can use the following query to locate both:

- `comments.author: ("Alice" AND "Bob") AND comments.text: "attack"`

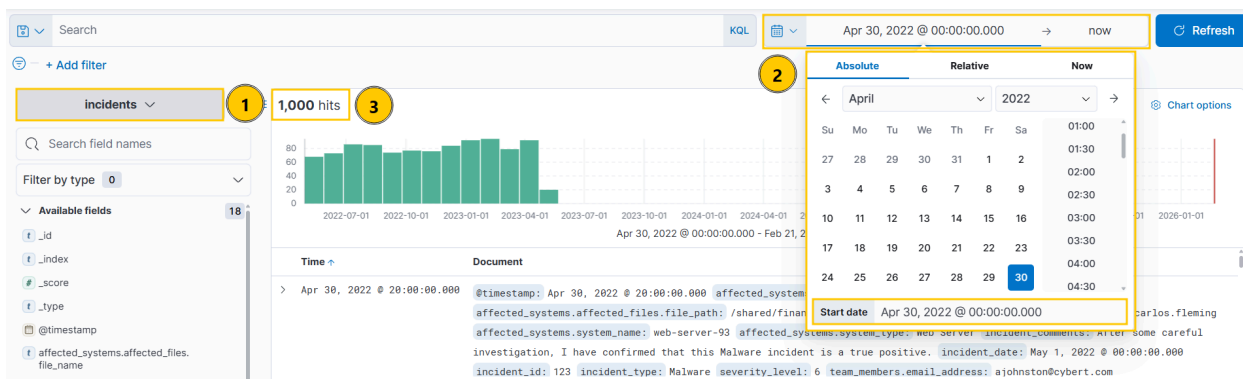
But, if you wanted to locate events with either **Alice** or **Bob**, you could use the **OR** operator instead:

- `comments.author: ("Alice" OR "Bob") AND comments.text: "attack"`



Querying a Larger Data Set: All of the queries in this room can be tested on a larger dataset within the **incidents** index pattern, which contains 1000 events. Let's switch indexes now and give it a shot to answer the questions for this task.

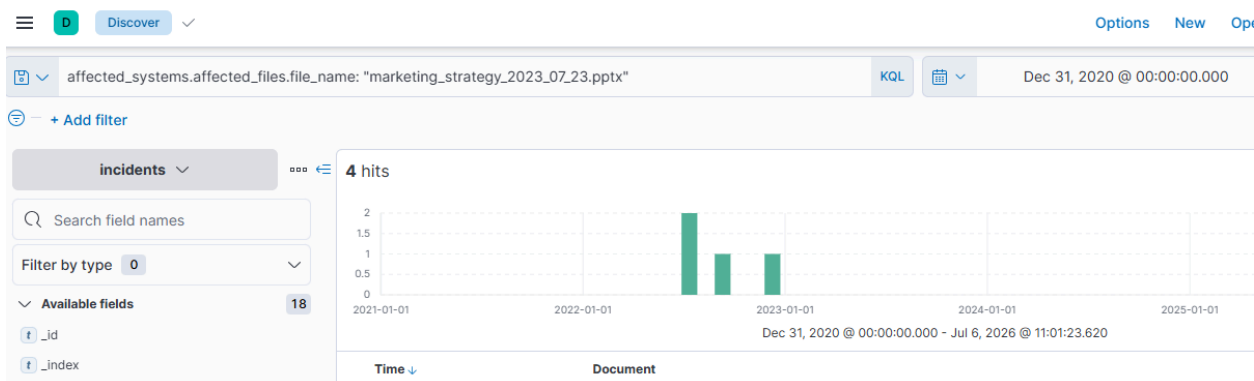
1. Select the **incidents** index pattern
2. Set your time range from **Apr 30, 2022 @ 00:00:00.000 → now**
3. View the results



Answer the questions below

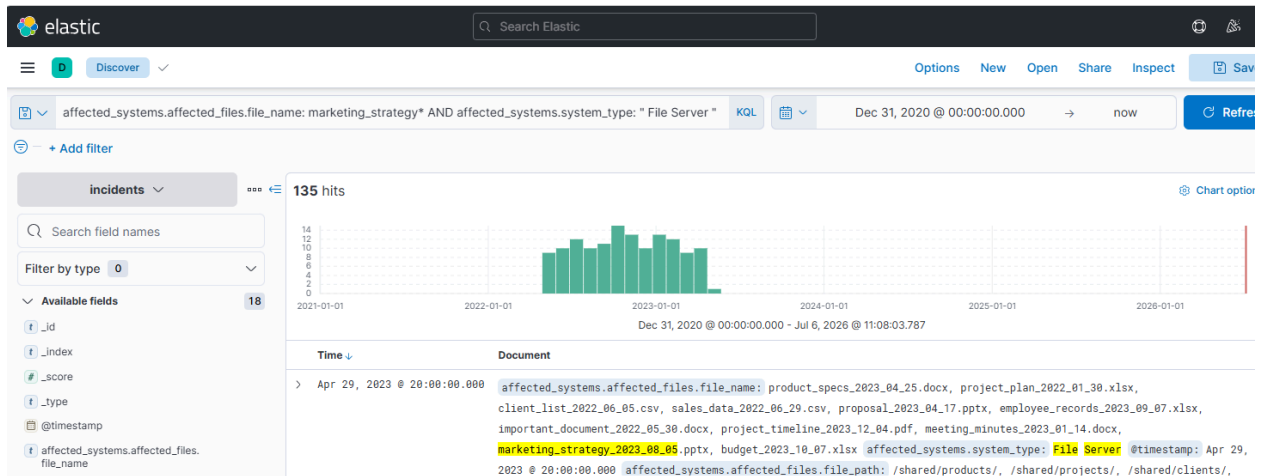
Review the available fields in the left-hand panel to build your first query. How many incidents involve the file `marketing_strategy_2023_07_23.pptx`?

4



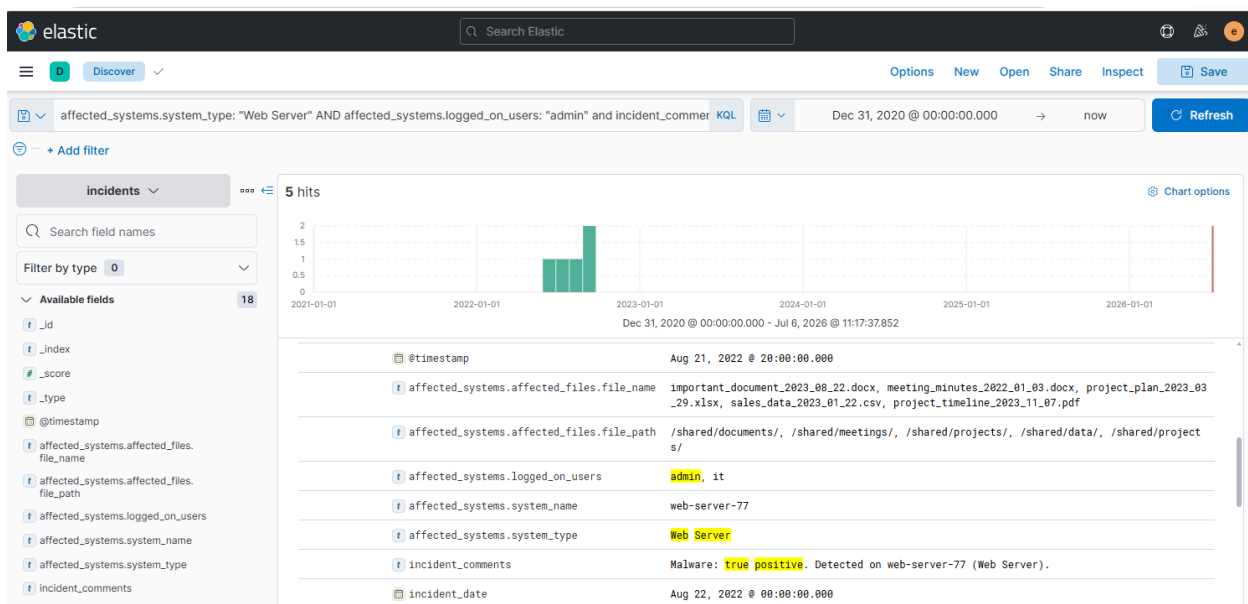
Try out a wildcard search to locate files using the `affected_systems.affected_files.file_name` field. How many incidents involve files on a File Server with names that start with `marketing_strategy`?

135



Which Web Server had a true positive incident where both the admin and it users were logged in?

web-server-77



Task 4 - Investigating With Regular Expressions

Regular expressions, often shortened to regex, allow you to use patterns to match field values. Instead of searching for exact words or using simple wildcards, regex enables you to define flexible, powerful search logic. You will encounter regular expressions frequently when working with log data, threat hunting queries, and pattern-based detection logic.

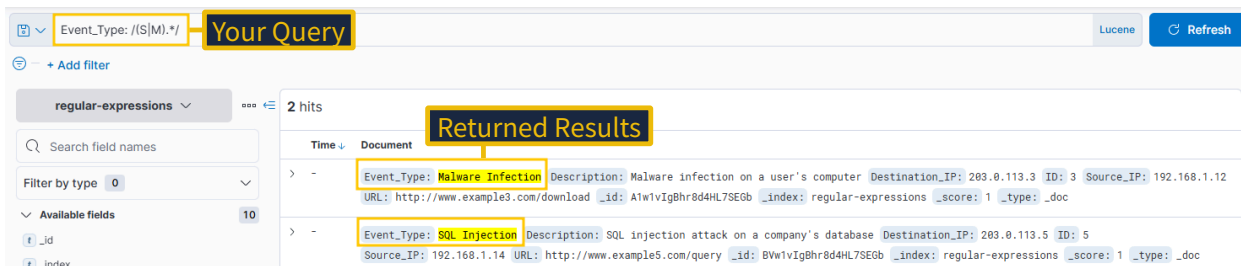
1. Ensure you've selected the `regular-expressions` index pattern
2. Turn off **KQL** to use the **Lucene** query syntax
3. Enter the query `Event_Type: /.*/`
4. View the results

When using regex in Lucene, the pattern must be wrapped in forward slashes `/`. In this case, the period `.` matches any single character, and `*` matches zero or more of the preceding character, so all results within the dataset will be returned.



Next, we may want to return field values from the `Event_Type` field that start with the letters `S` or `M`. To do so, we could use the following query:

- `Event_Type: /(S|M).*/`



Field Data Type Behavior: There is one important behavioral change worth mentioning regarding using regex searches against fields in Elastic. Depending on the data type of the field, `keyword`, or `text`, the results of your regex queries will vary.

So far, we have used regex searches on the `Event_Type` field, which has a data type of `keyword`. Regular expressions behave as you'd expect when searching fields

with the `keyword` data type. The behavior changes if the data type is `text`, like with the `Description` field. Try out the following query:

- `Description: /.*/`

So far, so good. All events within the index pattern are returned because we match all characters, regardless of length.

The screenshot shows the Elasticsearch Query Builder interface. The query bar contains 'Description: /.*/' and is labeled 'Your Query'. The left sidebar shows the 'Available fields' list with fields like _id, _index, _score, _type, Description, Destination_IP, Event_Type, ID, and Source_IP. The 'Filter by type' dropdown is set to '0'. The 'Returned Results' table shows 5 hits. Each hit has a 'Description' field highlighted in yellow, indicating a match. The descriptions are: 'Distributed denial of service attack on a company's website', 'Phishing email attempting to steal user credentials', 'Malware infection on a user's computer', 'Cross-site scripting attack on a web application', and 'SQL injection attack on a company's database'.

Now this is where things change. Try the following query and check the results:

- `Description: /(s|m).*/`

Notice that instead of the whole description being highlighted in yellow, only single words starting with the letters `s` or `m` are highlighted. This is because when a `text` field is analyzed, the string is tokenized, and the regular expression is matched against each word.

The screenshot shows the Elasticsearch Query Builder interface. The query bar contains 'Description: /(s|m).*/' and is labeled 'Your Query'. The left sidebar shows the 'Available fields' list with fields like _id, _index, _score, _type, Description, Destination_IP, Event_Type, ID, and Source_IP. The 'Filter by type' dropdown is set to '0'. The 'Returned Results' table shows 5 hits. Each hit has a 'Description' field highlighted in yellow, indicating a match. The descriptions are: 'Distributed denial of service attack on a company's website', 'Phishing email attempting to steal user credentials', 'Malware infection on a user's computer', 'Cross-site scripting attack on a web application', and 'SQL injection attack on a company's database'. In this view, only the words 'service', 'steal', 'Malware', 'site', and 'SQL' are highlighted in yellow, as they start with 's' or 'm'.

This approach allows flexibility, which can be further leveraged by combining it with additional expressions, as shown in the following query using the `AND` operator:

- `Description: /(s|m).*/ AND /user.*/`

Lucene Refresh

regular-expressions 2 hits

Search field names

Filter by type 0

Available fields 10

_id

_index

Returned Results

Time	Document
> -	Description: Phishing email attempting to steal user credentials Destination_IP: 203.0.113.2 Event_Type: Phishing ID: 2 Source_IP: 192.168.1.11 URL: http://www.example2.com/login _id: Alw1vIghr8d4HL7SEGb _index: regular-expressions _score: 2 _type: _doc
> -	Description: Malware infection on a user's computer Destination_IP: 203.0.113.3 Event_Type: Malware Infection ID: 3 Source_IP: 192.168.1.12 URL: http://www.example3.com/download _id: A1w1vIghr8d4HL7SEGb _index: regular-expressions _score: 2 _type: _doc

Answer the questions below

How many events exist where a client_list file was affected by ransomware?

69

elastic Search Elastic

Discover

Options New Open Share Inspect Save

incident_comments: ransomware AND client_list* KQL Dec 31, 2020 @ 00:00:00.000 → now Refresh

+ Add filter

incidents 69 hits Chart options

Search field names

Filter by type 0

Available fields 18

Popular

- # severity_level
- _id
- _index
- _score
- _type
- @timestamp
- affected_systems.affected_files.file_name
- affected_systems.affected_files.file_path
- affected_systems.logged_on_users

marketing_strategy_2023_09_17.pptx, product_specs_2022_10_06.docx, meeting_minutes_2022_06.docx, employee_records_2022_04_07.xlsx, client_list_2023_09_08.csv incident_comments: This is a Ransomware alert. After some investigation I have determined that this is a false negative. @timestamp: Apr 21, 2023 @ 20:00:00.000 affected_systems.affected_files.file_path: /shared/documents/, /shared/data/, /shared/marketing/, /shared/products/,

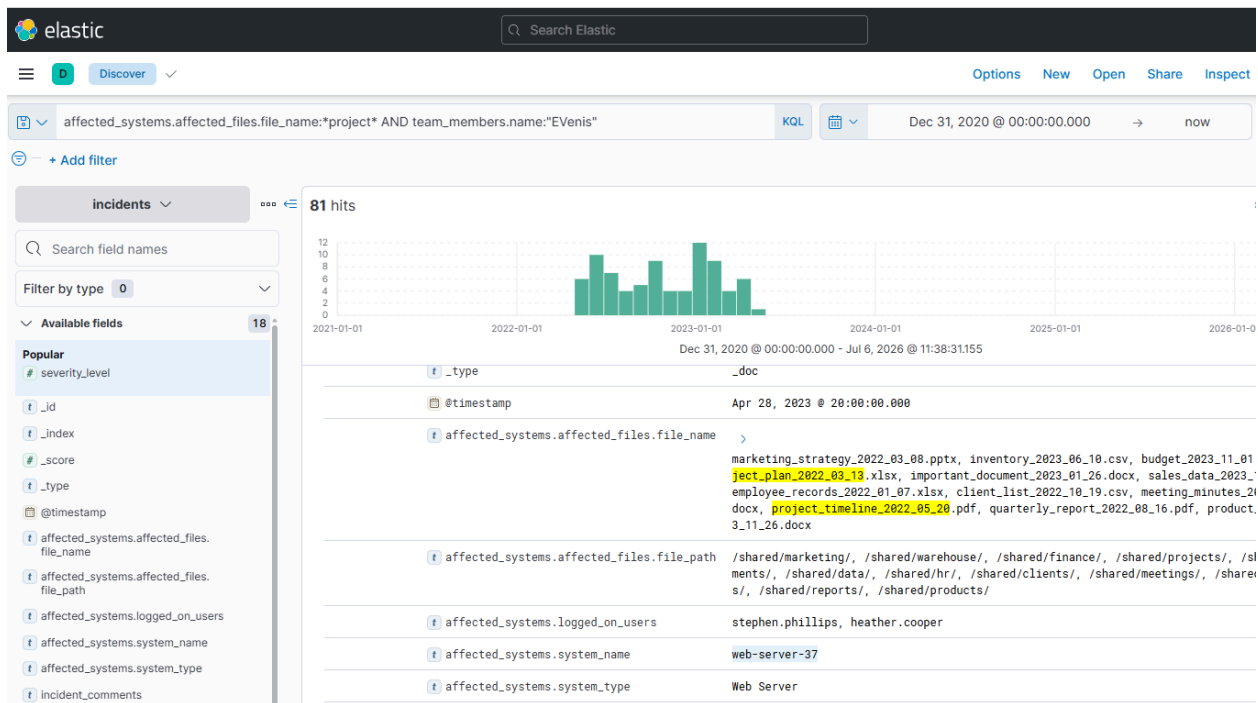
> Apr 14, 2023 @ 20:00:00.000 affected_systems.affected_files.file_name: employee_records_2022_12_09.xlsx, marketing_strategy_2023_09_26.pptx, important_document_2023_10_23.docx, project_plan_2022_09_07.xlsx, sales_data_2022_04_20.csv, meeting_minutes_2023_01_16.docx, budget_2023_04_25.xlsx, quarterly_report_2022_03_24.pdf, proposal_2023_03_26.pptx, client_list_2022_06_04.csv, inventory_2022_03_23.csv, project_timeline_2023_02_06.pdf, product_specs_2023_11_18.docx incident_comments: After some careful investigation, I have confirmed that this Ransomware incident is a true negative.

> Mar 25, 2023 @ 20:00:00.000 affected_systems.affected_files.file_name: employee_records_2022_08_29.xlsx, product_specs_2022_07_16.docx, quarterly_report_2022_04_07.pdf, inventory_2023_10_23.csv, important_document_2023_06_08.docx, proposal_2023_12_04.pptx,

Investigate the affected_systems.affected_files.file_name field using the regex */.project./*.

What is the affected system name on the latest event managed by EVENis?

web-server-37



Task 5 - Defining Ranges and Boundaries

Range queries allow us to search for documents whose field values fall within a specified range, making them ideal for numeric values, timestamps, and even IP addresses.

alert_id	alert_type	response_time_seconds
1	Malware Detection	120
2	Unusual Login Attempt	240
3	Suspicious Traffic	600
4	Unauthorized File Access	300
5	Phishing Email	180

Imagine you want to search for all documents where the `response_time_seconds` field value is greater than or equal to `100`. You could use the query `response_time_seconds >= 100`.

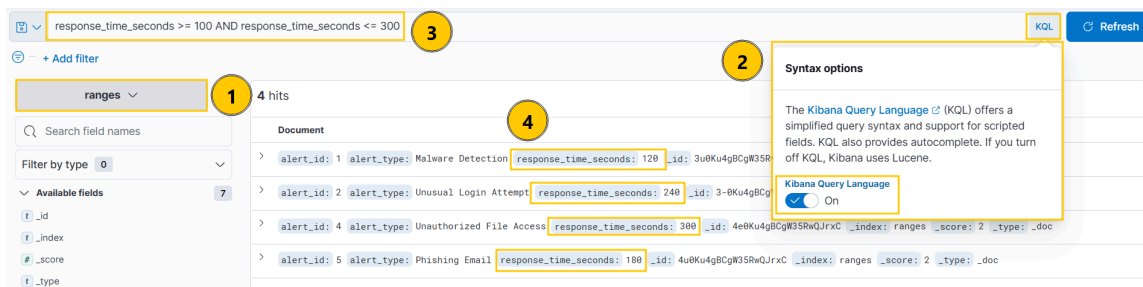
- This would return all entries from 1 to 5 since the value is greater than or equal to `100`.

Next, you may want to highlight values less than `300`. You could use the query `response_time_seconds < 300`.

- This would return `alert_id` 1, 2, and 5 since their value is less than `300`

Testing Out Ranged Queries: Now that you've seen the logic, let's see it in action. We'll use a specific index designed for testing these boundaries. You'll see that after entering the query, results with field values outside our specified range are not returned.

1. Select the `ranges` index pattern
2. Turn on **KQL**
3. Enter the query `response_time_seconds >= 100 AND response_time_seconds <= 300`
4. View the results

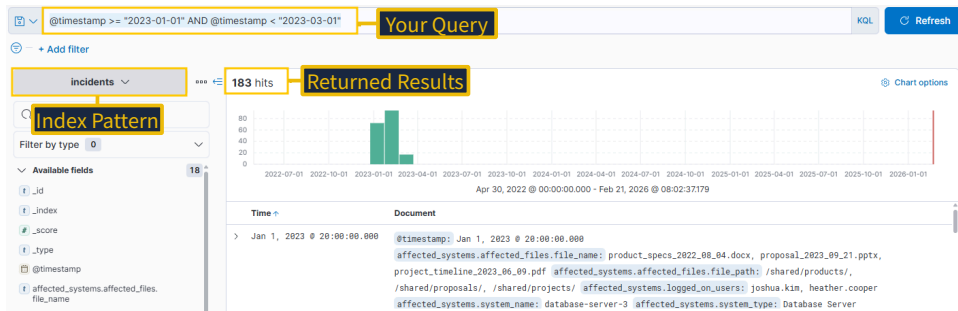


Using Ranges to Query Time: Ranges are beneficial for finding events that fall within specified dates and times. Let's switch back to the `incidents` index pattern to check it out. The events within this index pattern span a year from **April 2022** to **April 2023**. For example, if you want to locate events that occurred on or after `January 1, 2023`, you could use this query (note that we don't have to include hours, minutes, or seconds in our query):

- `@timestamp >= "2023-01-01"`

You could narrow it further to return only events before March 1, 2023, using the `AND` operator and both conditions, as shown below:

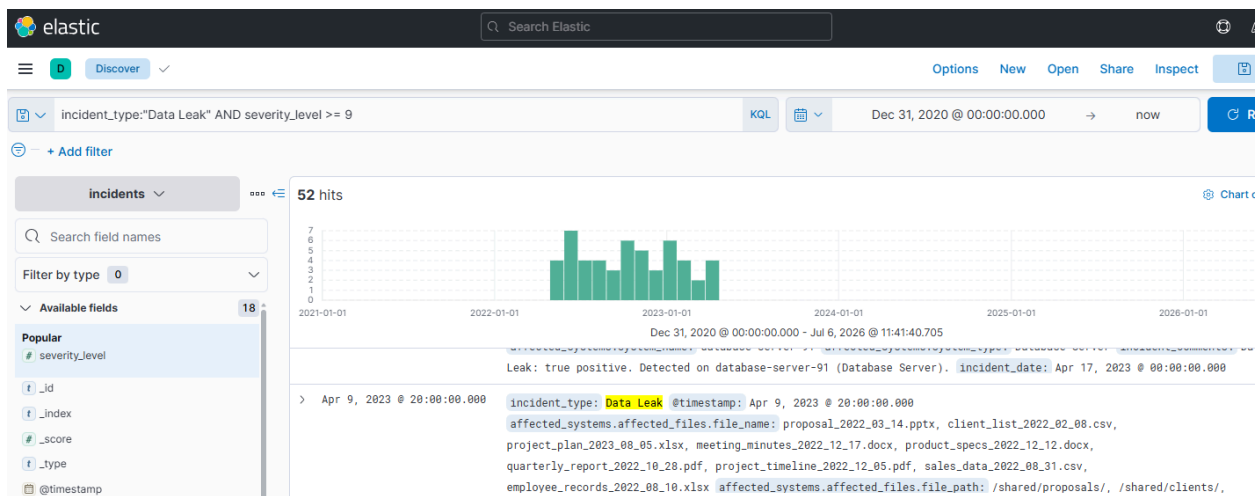
- `@timestamp >= "2023-01-01" AND @timestamp < "2023-03-01"`



Answer the questions below

How many Data Leak incidents have a severity_level of 9 or higher?

52



Check out the incidents investigated by AJohnston on or before December 1, 2022. How many events occurred on either an Email Server or a Web Server?

65

team_members.name:"AJohnston" AND affected_systems.system_type:"Email Server" OR affected_systems.system_type:"Web Server"

Investigate incident_id 1-500.

Which analyst stated that the Data Leak on file-server-65 was a false positive?

JLim

incident_comments: "data leak false positive. Detected on file-server" and
incident_id >= 1 AND incident_id <= 500

Task 6 - Searching Variants and Proximity

In real-world datasets, spelling inconsistencies are common, especially in user-submitted comments, analyst notes, or manually entered fields. A simple keyword search would miss these variations. This is where fuzzy searching becomes powerful. It allows us to find terms that are similar, but not identical, by specifying how many character differences are allowed between the search term and the stored value. For this task, we will use Lucene query syntax, as fuzzy searching is supported in Lucene but not in KQL. To demonstrate how this works, consider the following dataset.

host_name	status
server01	online
serber01	online
sirbir01	online
sorvor01	online

The tilde symbol `~` is the fuzzy operator, and the number that follows specifies the allowed edit distance (number of character changes). If you want to search for `host_name` field values similar to the term `server01`, you can use the following syntax:

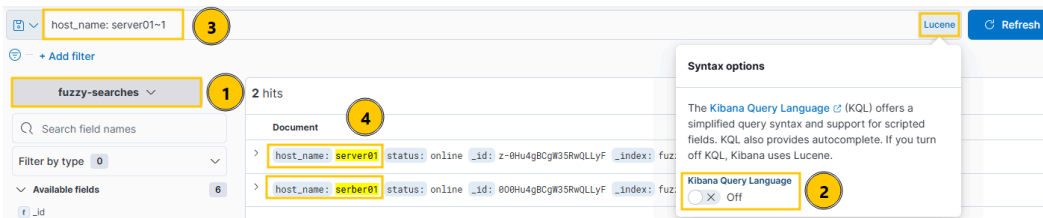
- `server01~1`: Allows 1 character difference and would return the values `server01` and `serber01`
- `server01~2`: Allows 2 character differences and would return the values `server01`, `serber01`, and `sorvor01`

Testing Fuzzy Searches

Let's head back to Kibana to try out the `~` operator with a simple fuzzy search.

1. Select the `fuzzy-searches` index pattern
2. Switch off **Kibana Query Language** to change to **Lucene**
3. Enter the query `host_name: server01~1`

4. View the results



Searching Proximity

Proximity searches with Lucene let you find documents whose field values contain two or more terms within a specified distance. The slop parameter sets the maximum distance between terms. For example, a slop value of `2` means that the words can be up to two positions away. To perform a proximity search, we can use the following syntax:

- `field_name: "search term"~slop_value`

As you can see, the tilde `~` character is used similarly to fuzzy searches, followed by a `slop_value`. The difference is that in proximity searches, the slop value is applied to a phrase enclosed in quotation marks `" "`. Let's consider the following dataset as an example.

log_id	log_message
1	Server error: failed login attempt
2	Login server - failed on startup with error
3	Login to server failed successfully
4	Server: Detected error in connection

To search for all documents where the terms `server` and `error` appear within a specified distance of 1 in the `log_message` field, you can use the following query:

- `log_message: "server error"~1`: Allows up to one position away and will return the values `Server error: failed login attempt`, and `Server: Detected error in connection`

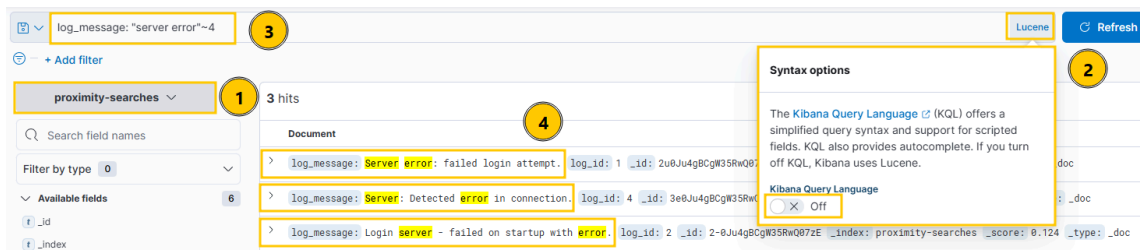
Please note that for analyzed text fields, punctuation like `:` and `-` are not stored as tokens and do not occupy positional increments; therefore, they do not count toward slop distance.

Testing Proximity Searches

Let's head back to Kibana to try out some proximity searches.

1. Switch to the `proximity-searches` index pattern
2. Ensure you're using the **Lucene** query syntax
3. Enter the query `log_message: "server error"~4`
4. View the results

A slop value of `4` allows the words `server` and `error` to appear up to four positional moves apart after analysis. The greater the slop value, the more flexibility Lucene allows in matching phrase order and distance.



Answer the questions below

Try out a fuzzy search to highlight variations in the `incident_comments` field. How many incidents has JLim handled in which the word `true` appears with a single character difference?

110

`team_members.name:JLim AND incident_comments:true~1`

Try out a proximity search on the `incident_comments` field. How many events contain `data-leak true-negative` within three words of each other?

33

`incident_comments:"data leak true negative"~3`

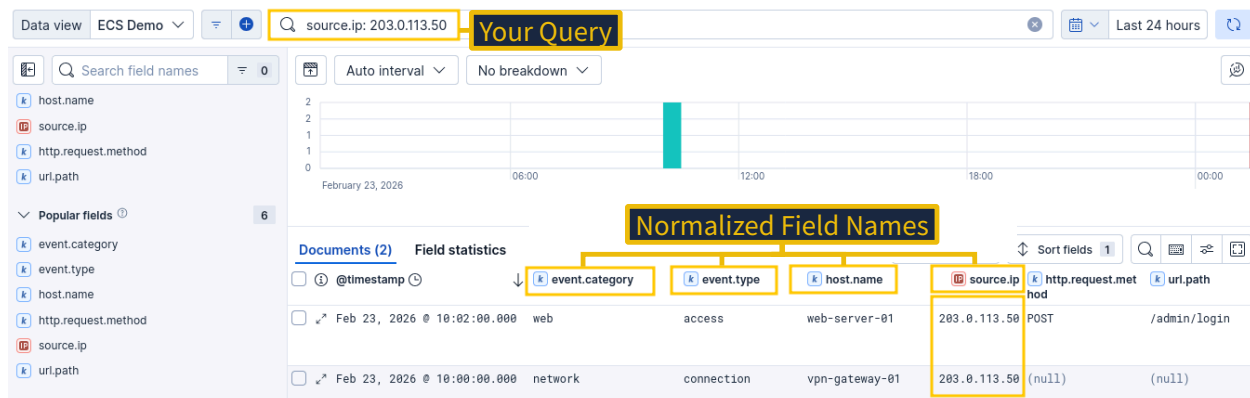
How many incidents handled by AJohnston contain `detected` and `negative` within two positions?

40

team_members.name:"AJohnston" AND incident_comments:"detected negative"~2

Task 7 - Elastic Query Architecture

Elastic Common Schema: [Elastic Common Schema\(opens in new tab\)](#) (ECS) is a standardized set of field-naming conventions designed to make searching and correlating data across different data sources easier. By normalizing how fields are structured and named, ECS enables consistent querying regardless of where the data originated. For example, imagine event data arriving from both a VPN server and a web server. Each source likely records the user's IP address when the user authenticates or sends a request. Without standardization, one log might use `src_ip` while another uses `client_address`. ECS solves this by normalizing both into a common field, such as `source.ip`, making cross-source searches and correlations significantly easier.



Event Query Language: [Event Query Language\(opens in new tab\)](#) (EQL) is a correlation-focused query language designed specifically for security use cases. Unlike traditional query languages that filter and search individual documents, EQL enables analysts to identify relationships between events over time. EQL is primarily used for detection engineering and threat hunting scenarios where identifying multi-step activity is critical. It allows analysts to write structured logic that connects activity across different sources. Because EQL operates on Elastic Common Schema (ECS) fields, it requires ECS-compliant data to function properly. In Elastic environments, EQL is most commonly used to power detection rules and is not run directly in Kibana, though it can be executed via the

Elasticsearch API. An example rule using EQL to monitor a Linux host for cron job modification may look something like this.

```
file where host.os.type == "linux" and
event.action in ("rename", "creation") and
file.path in (
  "/etc/crontab",
  "/etc/cron.allow",
  "/etc/cron.deny"
)
```

Answer the questions below

Which Elastic component standardizes field names across different data sources?

Elastic Common Schema

Which Elastic language is often used to write event correlation detection rules?

Task 8 - Conclusion

In this room, we explored advanced querying techniques in Kibana using the Kibana Query Language (KQL) and Lucene query syntax. You practiced filtering structured data, working with nested fields, applying wildcards, and leveraging regular expressions to extract meaningful insights from large datasets. These techniques allow analysts to move beyond basic searches and investigate data with greater precision, uncovering patterns, narrowing investigations, and improving analytical efficiency in real-world security scenarios.